

Task 1 — Steps in Static Malware Analysis

Steps	Description	Example
1. Identify the file	Determine the file type, size, and basic properties.	Check whether a file is a Windows .exe or Android .apk.
2. Calculate hashes	Generate MD5/SHA-1/SHA-256 hashes to uniquely identify the sample.	Calculate the SHA-256 hash of an APK and search it on VirusTotal.
3. Examine metadata	Look at timestamps, compiler information, package information, and other metadata.	For an APK, inspect package name, version, and permissions.
4. Analyze strings	Extract readable text that may reveal URLs, IPs, commands, filenames, or suspicious messages.	An EXE may contain URLs or PowerShell commands.
5. Check imports/functions	Examine APIs and libraries used by the program.	Windows APIs related to registry modification or process creation can be suspicious.
6. Analyze permissions	Particularly useful for Android APKs.	An app requesting SMS, contacts, microphone, and accessibility permissions may deserve investigation.
7. Disassemble/decompile	Convert executable code into assembly or higher-level representations.	Use tools such as JADX for APKs or Ghidra for EXEs.
8. Identify Indicators of	Record suspicious hashes, domains, IP	A suspicious domain embedded in an APK can

Compromise	addresses, filenames, registry keys, and other artifacts.	become an IoC.
------------	---	----------------

Task 2 — EICAR + Any.Run

The EICAR test file is specifically designed to safely test antivirus detection. It is not actual malware.

In Any.Run, look for observations such as:

1. Antivirus/security detection

- The EICAR file should be recognized as a test/detection artifact by security software.

Explanation:

- Security products identify the known EICAR test signature and report it as a test detection rather than allowing it to behave like normal software.

2. File creation/access

- The sandbox can record the presence or handling of the EICAR file.

Explanation:

- The file-system activity shows where the sample exists and which processes interact with it.

3. Process/security events

- The analysis environment may report security-related events generated when the sample is accessed.

Explanation:

- These events demonstrate how sandbox monitoring records activity around a suspicious/test file.

Task 3 — Cuckoo Sandbox

A Cuckoo report typically provides information such as:

Network activity

Look for:

- DNS requests
- Destination IP addresses
- HTTP/HTTPS connections
- Contacted domains
- Network protocols

Example report wording:

- The sandbox recorded DNS requests and outbound HTTP connections generated by the analyzed sample.

File activity

Look for:

- Files created
- Files modified
- Files deleted
- Temporary files
- Dropped executables

Example:

- The analysis showed that the sample created a temporary file in the user's application data directory.

Other useful artifacts

Cuckoo may also report:

- Processes created
- Registry activity
- Mutexes
- API calls
- Persistence mechanisms
- Screenshots
- Behavioral signatures

Task 4 — Static vs Dynamic Analysis

Static Analysis	Dynamic Analysis
-----------------	------------------

Examines the file without executing it.	Observes the file while it executes in a controlled environment.
Provides hashes, strings, metadata, imports, permissions, and code structure.	Provides processes, network connections, file changes, registry activity, and runtime behavior.
Can reveal capabilities that may not appear during one execution.	Shows what the sample actually does during the particular execution.

Three key differences

1. Execution

- Static analysis does not execute the malware, while dynamic analysis deliberately executes the sample inside an isolated sandbox.

2. Information obtained

- Static analysis focuses on the potential capabilities of the program; dynamic analysis focuses on its observed behavior.

3. Evasion

- Static analysis can sometimes reveal suspicious code even if the malware detects a sandbox. Dynamic analysis may miss behavior if the malware does not activate it during the particular analysis.

Task 5 — Recent Any.Run Malware Report

Two common anti-analysis techniques you may encounter are:

1. Virtualization/Sandbox Detection

- Malware can check for characteristics associated with virtual machines or analysis environments, such as specific processes, drivers, hardware identifiers, or system configuration.

Purpose:

- If the malware believes it is running inside a sandbox, it may stop malicious activity or behave differently.

2. Sleep/Delay

- Malware can intentionally wait before performing suspicious actions.

Purpose:

- Automated sandboxes often have limited execution times. Delaying activity can cause the malicious behavior to occur after the analysis has ended.

Other techniques you might encounter include:

- Debugger detection
- Environment checks
- User-interaction checks
- Obfuscated/encoded strings
- Process-name checks
- Timing checks